

```

1| // =====
2| // 文件: __init__.py
3| // =====
4| """Local K12 knowledge tracking pipeline."""
5|
6|
7| // =====
8| // 文件: __main__.py
9| // =====
10| from .cli import main
11|
12| raise SystemExit(main())
13|
14|
15| // =====
16| // 文件: cli.py
17| // =====
18| from __future__ import annotations
19|
20| import argparse
21| import json
22| from pathlib import Path
23|
24| from .materials import generate_materials
25| from .pipeline import LocalPipeline
26| from .reporting import generate_reports
27|
28|
29| def build_parser() -> argparse.ArgumentParser:
30|     parser = argparse.ArgumentParser(prog="edu-tracker")
31|     sub = parser.add_subparsers(dest="command", required=True)
32|
33|     manual = sub.add_parser("manual", help="process a manual question")
34|     manual.add_argument("--student-id", required=True)
35|     manual.add_argument("--subject", required=True)
36|     manual.add_argument("--grade", required=True)
37|     manual.add_argument("--text", required=True)
38|     manual.add_argument("--answer", default="")
39|     manual.add_argument("--student-answer", default="")
40|
41|     photo = sub.add_parser("photo", help="process a photo question")
42|     photo.add_argument("--student-id", required=True)
43|     photo.add_argument("--subject", required=True)
44|     photo.add_argument("--grade", required=True)
45|     photo.add_argument("--image-path", required=True)
46|     photo.add_argument("--ocr-text", default="")
47|     photo.add_argument("--answer", default="")
48|     photo.add_argument("--student-answer", default="")
49|
50|     report = sub.add_parser("report", help="generate parent/student markdown reports")
51|     report.add_argument("--student-id", default="")
52|
53|     materials = sub.add_parser("materials", help="generate copyright materials")
54|
55|     return parser
56|
57|
58| def main() -> int:
59|     parser = build_parser()
60|     args = parser.parse_args()
61|     project_root = Path(__file__).resolve().parents[2]
62|     pipeline = LocalPipeline(project_root/"教育智能体")
63|
64|     if args.command == "manual":
65|         result = pipeline.process_manual_question(
66|             student_id=args.student_id,
67|             subject=args.subject,
68|             grade=args.grade,
69|             text=args.text,
70|             answer=args.answer,
71|             student_answer=args.student_answer,
72|         )
73|         print(json.dumps(result, ensure_ascii=False, indent=2))
74|         return 0

```

```

75|
76| if args.command == "photo":
77|     result = pipeline.process_photo(
78|         student_id=args.student_id,
79|         subject=args.subject,
80|         grade=args.grade,
81|         image_path=args.image_path,
82|         ocr_text=args.ocr_text,
83|         answer=args.answer,
84|         student_answer=args.student_answer,
85|     )
86|     print(json.dumps(result, ensure_ascii=False, indent=2))
87|     return 0
88|
89| if args.command == "report":
90|     sid = args.student_id.strip() or None
91|     generated = generate_reports(project_root, sid)
92|     print(json.dumps({"generated": [str(path) for path in generated]}, ensure_ascii=False, indent=2))
93|     return 0
94|
95| if args.command == "materials":
96|     generated = generate_materials(project_root)
97|     print(json.dumps({"generated": [str(path) for path in generated]}, ensure_ascii=False, indent=2))
98|     return 0
99|
100| return 1
101|
102|
103| if __name__ == "__main__":
104|     raise SystemExit(main())
105|
106| // =====
107| // 文件: knowledge_registry.py
108| // =====
109| from __future__ import annotations
110|
111| from dataclasses import dataclass
112|
113| from .models import KnowledgePoint
114|
115|
116| @dataclass
117| class MatchResult:
118|     point: KnowledgePoint
119|     confidence: float
120|     reason: str
121|
122|
123| def registry() -> list[KnowledgePoint]:
124|     return [
125|         KnowledgePoint(
126|             point_code="MATH-PRI-G2-NS-001",
127|             subject="数学",
128|             subject_code="MATH",
129|             stage="小学",
130|             stage_code="PRI",
131|             grade="二年级",
132|             topic="数与代数",
133|             topic_code="NS",
134|             standard_version="2022",
135|             tracking_mode="BKT",
136|         ),
137|         KnowledgePoint(
138|             point_code="MATH-PRI-G2-NS-002",
139|             subject="数学",
140|             subject_code="MATH",
141|             stage="小学",
142|             stage_code="PRI",
143|             grade="二年级",
144|             topic="数与代数",
145|             topic_code="NS",
146|             standard_version="2022",
147|             tracking_mode="BKT",
148|         ),

```

```

149| KnowledgePoint(
150|   point_code="MATH-PRI-G2-NS-003",
151|   subject="数学",
152|   subject_code="MATH",
153|   stage="小学",
154|   stage_code="PRI",
155|   grade="二年级",
156|   topic="数与代数",
157|   topic_code="NS",
158|   standard_version="2022",
159|   tracking_mode="BKT",
160| ),
161| KnowledgePoint(
162|   point_code="ENG-JHS-G8-GRM-001",
163|   subject="英语",
164|   subject_code="ENG",
165|   stage="初中",
166|   stage_code="JHS",
167|   grade="八年级",
168|   topic="语法",
169|   topic_code="GRM",
170|   standard_version="2022",
171|   tracking_mode="MIXED",
172| ),
173| KnowledgePoint(
174|   point_code="CHN-PRI-G2-READ-001",
175|   subject="语文",
176|   subject_code="CHN",
177|   stage="小学",
178|   stage_code="PRI",
179|   grade="二年级",
180|   topic="阅读与鉴赏",
181|   topic_code="READ",
182|   standard_version="2022",
183|   tracking_mode="MIXED",
184| ),
185| ]
186|
187|
188| def match_question(subject: str, grade: str, text: str) -> MatchResult:
189|     candidates = registry()
190|     normalized = f"{subject} {grade} {text}"
191|
192|     if "乘法口诀" in normalized or "乘法" in normalized:
193|         point = next(p for p in candidates if p.point_code == "MATH-PRI-G2-NS-003")
194|         return MatchResult(point=point, confidence=0.94, reason="命中乘法关键词")
195|     if "阅读" in normalized or "短文" in normalized:
196|         point = next(p for p in candidates if p.point_code == "CHN-PRI-G2-READ-001")
197|         return MatchResult(point=point, confidence=0.72, reason="命中阅读关键词")
198|     if "过去时" in normalized or "比较级" in normalized:
199|         point = next(p for p in candidates if p.point_code == "ENG-JHS-G8-GRM-001")
200|         return MatchResult(point=point, confidence=0.81, reason="命中英语语法关键词")
201|
202|     point = candidates[0]
203|     return MatchResult(point=point, confidence=0.35, reason="未命中规则，使用默认候选")
204|
205| // =====
206| // 文件: materials.py
207| // =====
208| from __future__ import annotations
209|
210| from collections import Counter
211| from datetime import datetime, timezone
212| from pathlib import Path
213| from typing import List
214|
215| from .reporting import load_jsonl
216|
217|
218| def now_date() -> str:
219|     return datetime.now(timezone.utc).strftime("%Y-%m-%d")
220|
221|
222| def build_system_overview(code_dir: Path) -> str:

```

```

223| # Gather module stats
224| modules = []
225| for f in sorted(code_dir.rglob("*.py")):
226|     if "__pycache__" in str(f):
227|         continue
228|     lines = len(f.read_text().splitlines())
229|     modules.append((f.relative_to(code_dir.parent), lines))
230| total_lines = sum(m[1] for m in modules)
231|
232| module_table = "\n".join(
233|     f"| {m[0]} | {m[1]} |" for m in modules
234| )
235|
236| return f"""---
237| title: 系统说明
238| tags:
239| - project/k12-tracking
240| - copyright/materials
241| updated: {now_date()}
242| ---
243|
244| # 系统说明
245|
246| ## 一、软件名称
247|
248| K12 教育知识点掌握程度跟踪系统
249|
250| ## 二、软件用途
251|
252| 本系统面向家庭场景，由家长主导，帮助家长和学生追踪各学科知识点的掌握情况。核心解决三大痛点：
253|
254| 1. **信息黑箱**：家长不清楚孩子到底哪些知识点掌握了、哪些薄弱。
255| 2. **时间匮乏**：家长没有精力系统梳理 K12 知识体系和逐题诊断。
256| 3. **数据碎片**：试卷、作业分布在纸质和不同 App 中，无法跨平台整合分析。
257|
258| ## 三、目标用户
259|
260| - **主要用户**：K12 阶段学生的家长（尤其是小学、初中阶段）
261| - **次要用户**：学生本人（查看掌握进度和复盘建议）
262| - **适用角色**：家庭场景，非学校/SaaS 规模化部署
263|
264| ## 四、系统架构
265|
266| 本系统采用 **本地优先 (Local-First)** 架构，所有数据存储和分析均在用户本地 macOS 环境中完成，不依赖于任何云服务。
267|
268| ```
269|
270|
271|
272|
273|
274|
275|
276|
277|
278|
279|
280|
281|
282|
283|
284|
285|
286|
287|
288|
289|
290|
291|
292|
293|
294|
295|
296|

```

```
297| |
298| | | questions | | mastery | | evidence | |
299| | | .jsonl | | .jsonl | | .jsonl | |
300| | |
301| | |
302| | | ingest.jsonl | | audit.jsonl | |
303| | |
304| |
305| | ``
306|
307| ## 五、模块说明
308|
309| | 模块 | 行数 | 职责 |
310| | :--- | :---: | :--- |
311| {module_table}
312|
313| ### 5.1 核心分析管线 (pipeline.py)
314|
315| `pipeline.py` 是系统的核心处理引擎，负责：
316|
317| - **录入接收**：接收手动输入或照片导入的题目数据。
318| - **知识点映射**：将题目文本与课标知识库进行匹配，确定所属学科、年级、知识点。
319| - **对错分流**：根据学生答案与标准答案比对，判断正确/错误，生成两套不同的处理逻辑：
320| - **正确题**：增加掌握置信度，降低该知识点的复习优先级。
321| - **错题**：记录为「待复核」，触发薄弱标记，降低掌握度。
322| - **掌握度更新**：基于正确/错误历史，通过间隔重复和 BKT（贝叶斯知识追踪）混合策略更新知识点掌握状态。
323| - **审计留痕**：每次操作生成审计事件，写入 `audit.jsonl`。
324|
325| ### 5.2 结果视图生成 (reporting.py)
326|
327| `reporting.py` 负责将追踪数据转化为可视化的 Markdown 报告页面：
328|
329| - **家长端总览**：聚合展示各学科掌握概况、短板列表、最近题目、待复核项。
330| - **学生周报**：面向学生的每周摘要页，展示当前掌握度变化、推荐复习的知识点。
331| - **题目详情页**：每题一条记录，展示原题内容、学生答案、标准答案、映射知识点。
332|
333| ### 5.3 数据模型 (models.py)
334|
335| 定义系统核心领域对象：
336|
337| - **KnowledgePoint**：知识点实体，包含学科、年级、主题、掌握度、更新时间等字段。
338| - **QuestionRecord**：题目记录，包含题目文本、答案、学生答案、映射知识点 ID、题目类型等。
339| - **EvidenceEvent**：证据事件，记录每次掌握度变化的来源（题目/手动修正）。
340| - **MasteryState**：掌握状态快照，记录每个知识点在某一时刻的掌握概率。
341| - **AuditEvent**：审计日志，记录每次关键操作的原始内容。
342|
343| ### 5.4 知识点注册 (knowledge_registry.py)
344|
345| 管理知识点注册表和学科配置：
346|
347| - 支持按学科区分追踪策略：BKT（词汇/语法类记忆型学科）与 RULE（数学/科学类规则型学科）。
348| - 提供知识点查找和学科列表查询接口。
349|
350| ### 5.5 CLI 入口 (cli.py)
351|
352| 命令行接口，支持子命令：
353|
354| | 子命令 | 功能 |
355| | :--- | :--- |
356| | `manual` | 手动录题 |
357| | `photo` | 照片导入并处理 |
358| | `report` | 生成结果视图 |
359| | `materials` | 生成软著申报材料 |
360|
361| ### 5.6 软著材料生成 (materials.py)
362|
363| 自动化生成软著申报所需的文档材料，包括：
364|
365| - 系统说明
366| - 用户操作手册
367| - 安装部署说明
368| - 版本说明
369| - 日志索引
370| - 源代码首页提取
```

```
371|
372| ## 六、技术栈
373|
374| | 类别 | 选择 | 说明 |
375| | :--- | :--- | :--- |
376| | 语言 | Python 3.9+ | 跨平台，生态丰富 |
377| | 命令行 | argparse | Python 标准库 |
378| | 数据存储 | JSONL | 追加日志，适合本地场景 |
379| | 文档输出 | Markdown | 易于阅读和版本管理 |
380| | 知识库 | Obsidian Vault | 支持双向链接与图谱 |
381| | 构建工具 | Makefile | 一键执行常见任务 |
382|
383| ## 七、数据流
384|
385| ### 手动录题流程
386|
387| ```
388| 用户输入题目 → CLI 解析 → pipeline.manual_ingest()
389| → 知识点映射 → 对错分流
390| └─ 正确题 → 增加掌握置信度
391| └─ 错题 → 标记待复核 → 降低掌握度
392| → 写入 questions.jsonl
393| → 写入 mastery.jsonl
394| → 写入 evidence.jsonl
395| → 写入 audit.jsonl
396| → 完成
397| ```
398|
399| ### 照片导入流程
400|
401| ```
402| 用户提供照片+OCR文本 → CLI 解析 → pipeline.photo_ingest()
403| → 旁路文本验证（当前阶段）→ 同分流更新
404| → 写入 ingest.jsonl + questions.jsonl
405| → 同上掌握更新与日志
406| ```
407|
408| ### 报告生成流程
409|
410| ```
411| 用户执行 report → reporting.generate_views()
412| → 读取所有 JSONL → 聚合分析
413| → 生成家长端总览（05-结果视图/家长端总览.md）
414| → 生成学生周报（05-结果视图/学生端-s*.md）
415| → 生成题目详情（05-结果视图/题目详情/q_*.md）
416| ```
417|
418| ## 八、安全与隐私
419|
420| - **本地优先**：所有数据存储在用户本地 macOS 中，不上传任何云端。
421| - **无数据收集**：系统不收集用户使用行为数据。
422| - **审计可追溯**：所有数据变更通过 audit.jsonl 完整记录。
423|
424| ## 九、合规说明
425|
426| - 符合《个人信息保护法》对未成年人信息保护的合规框架设计。
427| - 用户协议、隐私政策、监护人授权书等法律文档随项目提供。
428| """
429|
430|
431| def build_user_manual() -> str:
432|     return f"""---
433| title: 用户操作手册
434| tags:
435|   - project/k12-tracking
436|   - copyright/materials
437| updated: {now_date()}
438| ---
439|
440| # 用户操作手册
441|
442| ## 一、安装环境
443|
444| ### 1.1 前置依赖
```

```
445|
446| - macOS 操作系统（推荐 macOS 12+）
447| - Python 3.9 或更高版本
448| - Obsidian（用于查看结果视图和知识图谱，可选）
449|
450| ### 1.2 验证安装
451|
452| ```bash
453| python3 --version
454| # 应输出 Python 3.9.x 或更高
455|
456| git --version
457| # 应输出 git 2.x 或更高
458| ```
459|
460| ### 1.3 项目结构确认
461|
462| ```
463| 教育智能体/
464| |—— edu_tracker/      # Python 包
465| |—— services/         # 核心服务
466| |—— 教育智能体/       # Obsidian Vault
467| |   |—— 05-结果视图/   # 生成的结果视图
468| |   |—— 09-软著材料/   # 软著材料输出
469| |   |—— logs/         # 日志文件
470| |—— Makefile          # 一键任务
471| |—— README.md         # 项目说明
472| ```
473|
474| ## 二、家长使用指南
475|
476| ### 2.1 使用流程概览
477|
478| ```
479| 录题 → 分析 → 查看结果 → 针对性辅导 → 再次录题 → 追踪进步
480| ```
481|
482| ### 2.2 手动录题
483|
484| 手动录题是最基本的录入方式。家长将试卷或练习题中的题目内容、标准答案和学生答案通过命令行输入。
485|
486| **命令模板：**
487|
488| ```bash
489| python3 -m edu_tracker manual \
490| --student-id s001 \
491| --subject<学科> \
492| --grade<年级> \
493| --text"<题目文字>" \
494| --answer"<标准答案>" \
495| --student-answer"<学生答案>"
496| ```
497|
498| **参数说明：**
499|
500| | 参数 | 必填 | 说明 | 示例 |
501| | :--- | :---: | :--- | :--- |
502| | `--student-id` | 是 | 学生编号 | `s001` |
503| | | `--subject` | 是 | 学科 | `数学`、`语文`、`英语` |
504| | | `--grade` | 是 | 年级 | `二年级`、`三年级` |
505| | | `--text` | 是 | 题目文字内容 | `3 × 4 = ?` |
506| | | `--answer` | 是 | 标准答案 | `12` |
507| | | `--student-answer` | 是 | 学生答案 | `12` 或 `15` |
508|
509| **示例：录入一道乘法题**
510|
511| ```bash
512| python3 -m edu_tracker manual \
513| --student-id s001 \
514| --subject 数学 \
515| --grade 二年级 \
516| --text "3 × 4 = ?" \
517| --answer "12" \
518| --student-answer "12"

```

```
519| ```
520|
521| 系统输出：
522| - 题目记录写入 `logs/questions.json`
523| - 掌握度更新写入 `logs/mastery.json`
524| - 证据事件写入 `logs/evidence.json`
525|
526| ### 2.3 照片导入
527|
528| 照片导入适用于已经完成的纸质试卷或练习册。当前阶段支持旁路文本验证（OCR 文本由家长手动输入）。
529|
530| **命令模板：**
531|
532| ```bash
533| python3 -m edu_tracker photo \
534| --student-id s001 \
535| --subject<学科> \
536| --grade<年级> \
537| --image-path<图片路径> \
538| --ocr-text "<OCR 识别文本>" \
539| --answer "<标准答案>" \
540| --student-answer "<学生答案>"
541| ```
542|
543| **参数说明：**
544|
545| | 参数 | 必填 | 说明 | 示例 |
546| | :--- | :---: | :--- | :--- |
547| | `--image-path` | 是 | 试卷照片文件路径 | `/path/to/paper.jpg` |
548| | `--ocr-text` | 是 | 题目文本（当前手动输入） | `"3 × 4 = ?"` |
549|
550| **示例：**
551|
552| ```bash
553| python3 -m edu_tracker photo \
554| --student-id s001 \
555| --subject 数学 \
556| --grade 二年级 \
557| --image-path ./试卷/乘法练习1.jpg \
558| --ocr-text "3 × 4 = ?" \
559| --answer "12" \
560| --student-answer "12"
561| ```
562|
563| ### 2.4 生成结果视图
564|
565| 录完题目后，执行报告生成命令：
566|
567| ```bash
568| python3 -m edu_tracker report
569| ```
570|
571| 系统将在 `05-结果视图` 目录下生成：
572| - `家长端总览.md` — 学科掌握概况、短板列表、最近题目
573| - `学生端-s001.md` — 面向学生的周报
574| - `题目详情/q_xxxxxxxxxx.md` — 每道题的详细分析
575|
576| ### 2.5 查看分析结果
577|
578| 家长端总览包含：
579|
580| 1. **学科掌握度摘要**：按学科、知识主题展示平均掌握率。
581| 2. **短板知识点**：掌握度低于阈值的知识点清单，按紧急程度排序。
582| 3. **最近录入题目**：最近录入的题目列表，标注正确/错误状态。
583| 4. **待复核项**：判定为「边缘正确」或「答案有歧义」的题目，需家长人工确认。
584|
585| ### 2.6 生成软著材料
586|
587| ```bash
588| python3 -m edu_tracker materials
589| ```
590|
591| 在 `09-软著材料/生成稿` 目录下生成/更新：
592| - 系统说明
```


593 | - 用户操作手册
594 | - 安装部署说明
595 | - 版本说明
596 | - 日志索引
597 | - 源代码页
598 |
599 | ## 三、学生使用指南
600 |
601 | ### 3.1 周报查看
602 |
603 | 学生通过 Obsidian 打开生成的 `05-结果视图/学生端-s001.md` 查看每周学习总结。
604 |
605 | ### 3.2 周报内容
606 |
607 | 1. **本周掌握变化**：各知识点掌握度对比上周的变化。
608 | 2. **当前短板清单**：掌握度低于 60% 的知识点，按优先级排列。
609 | 3. **推荐复习知识点**：系统根据遗忘曲线推荐的 3-5 个复习知识点。
610 | 4. **原题回溯**：每个薄弱知识点可跳转到对应题目的详情页，查看原始题目内容。
611 |
612 | ## 四、常见问题
613 |
614 | ### 4.1 录题后没看到掌握变化？
615 |
616 | 首次录题时系统会建立初始掌握状态，可能需要 1-2 道题后才能看到趋势变化。建议连续录入 3-5 道同一知识点的题目以获得有意义的掌握度变化。
617 |
618 | ### 4.2 如何修改录错的题目？
619 |
620 | 当前版本暂不支持直接编辑已录入的题目。建议重新录入正确的版本，系统会合并分析。
621 |
622 | ### 4.3 支持哪些学科？
623 |
624 | 系统基于 2022 版国家课程标准，涵盖 15 个学科，包括：语文、数学、英语、科学、道德与法治、体育与健康、艺术、劳动、信息科技、思想政治、历史、地理、物理、化学、生物。
625 |
626 | ### 4.4 数据安全吗？
627 |
628 | 所有数据存储在本地 macOS 中，不连接任何云端服务。系统不收集任何用户数据，审计日志供用户自行追溯。
629 |
630 | ## 五、注意事项
631 |
632 | 1. 照片导入当前阶段 OCR 文本需手动输入，正式 OCR 接入将在后续版本完成。
633 | 2. 命令参数中的空格和特殊字符建议用引号包裹。
634 | 3. 首次使用建议先通过手动录题熟悉流程，再尝试照片导入。
635 | 4. 建议每周至少录入 1-2 次题目以保持掌握度追踪的连续性。
636 | """"
637 |
638 |
639 | def build_installation_guide() -> str:
640 | return f"""---
641 | title: 安装部署说明
642 | tags:
643 | - project/k12-tracking
644 | - copyright/materials
645 | updated: {now_date()}
646 | ---
647 |
648 | # 安装部署说明
649 |
650 | ## 一、环境要求
651 |
652 | ### 1.1 操作系统
653 |
654 | - macOS 12 (Monterey) 或更高版本
655 | - 推荐 macOS 14+ (Apple Silicon / arm64)
656 | - 尚未测试 Windows/Linux 环境，后续版本考虑支持
657 |
658 | ### 1.2 运行时
659 |
660 | | 依赖 | 最低版本 | 说明 |
661 | | :--- | :---: | :--- |
662 | | Python | 3.9 | 推荐 3.11+ |
663 | | pip | 21+ | Python 包管理器 |
664 | | Git | 2.x | 版本管理与发布 |
665 | | Make | 3.x | 任务编排 |
666 |

```
667| ### 1.3 可选依赖
668|
669| | 依赖 | 用途 |
670| | :--- | :--- |
671| | Obsidian | 查看 Markdown 结果视图和知识图谱 |
672| | pandoc | 将 Markdown 导出为 PDF/Word 格式 |
673|
674| ## 二、项目路径说明
675|
676| | 目录 | 路径 | 用途 |
677| | :--- | :--- | :--- |
678| | 项目根目录 | `/Users/mac/Documents/教育智能体` | 代码与配置 |
679| | Obsidian Vault | `/Users/mac/Documents/教育智能体/教育智能体` | 知识库与结果视图 |
680| | 日志目录 | `教育智能体/logs/` | JSONL 数据文件 |
681| | 结果视图 | `教育智能体/05-结果视图/` | 生成的报告 |
682|
683| ## 三、安装步骤
684|
685| ### 3.1 获取代码
686|
687| ```bash
688| cd /Users/mac/Documents/教育智能体
689| ```
690|
691| 项目已完整包含代码和 Obsidian Vault，无需额外克隆。
692|
693| ### 3.2 验证运行环境
694|
695| ```bash
696| python3 --version
697| # 应输出 Python 3.9.x 或更高
698| ```
699|
700| ### 3.3 运行验证
701|
702| ```bash
703| make check
704| # 执行：检查 Python 版本、目录完整性、日志文件、结果视图生成
705| ```
706|
707| ## 四、运行方式
708|
709| ### 4.1 手动录题
710|
711| ```bash
712| python3 -m edu_tracker manual \
713| --student-id s001 \
714| --subject 数学 \
715| --grade 二年级 \
716| --text "乘法口诀练习" \
717| --answer "6" \
718| --student-answer "6"
719| ```
720|
721| ### 4.2 照片导入
722|
723| ```bash
724| python3 -m edu_tracker photo \
725| --student-id s001 \
726| --subject 数学 \
727| --grade 二年级 \
728| --image-path /path/to/paper.jpg \
729| --ocr-text "乘法口诀练习" \
730| --answer "6" \
731| --student-answer "6"
732| ```
733|
734| ### 4.3 生成结果视图
735|
736| ```bash
737| python3 -m edu_tracker report
738| ```
739|
740| ### 4.4 生成软著材料
```

```

741|
742| ```bash
743| python3 -m edu_tracker materials
744| # 或
745| make materials
746| ```
747|
748| ## 五、输出目录说明
749|
750| | 输出 | 路径 | 格式 |
751| | :--- | :--- | :--- |
752| | 题目记录 | `logs/questions.jsonl` | JSONL |
753| | 掌握状态 | `logs/mastery.jsonl` | JSONL |
754| | 证据事件 | `logs/evidence.jsonl` | JSONL |
755| | 导入记录 | `logs/ingest.jsonl` | JSONL |
756| | 审计日志 | `logs/audit.jsonl` | JSONL |
757| | 家长端总览 | `05-结果视图/家长端总览.md` | Markdown |
758| | 学生周报 | `05-结果视图/学生端-s*.md` | Markdown |
759| | 题目详情 | `05-结果视图/题目详情/q_*.md` | Markdown |
760| | 系统说明 | `09-软著材料/生成稿/系统说明.md` | Markdown |
761| | 用户手册 | `09-软著材料/生成稿/用户操作手册.md` | Markdown |
762| | 安装说明 | `09-软著材料/生成稿/安装部署说明.md` | Markdown |
763| | 版本说明 | `09-软著材料/生成稿/版本说明.md` | Markdown |
764|
765| ## 六、一键任务 (Makefile)
766|
767| | 命令 | 功能 |
768| | :--- | :--- |
769| | `make check` | 运行完整性检查 |
770| | `make report` | 生成结果视图 |
771| | `make materials` | 生成软著材料 |
772| | `make bundle` | 打包迁移包到 `dist/` |
773| | `make init-repo` | 初始化独立仓库 |
774| | `make release VERSION=x.y.z` | 生成发布说明 |
775| | `make tag VERSION=x.y.z` | 创建 Git 标签 |
776| | `make all` | 完整流程 (check → report → materials) |
777| | ""
778|
779|
780| def build_version_note(questions: List[dict], mastery_rows: List[dict], ingest_rows: List[dict]) -> str:
781| subject_counter = Counter(row.get("subject", "未知") for row in questions)
782| subject_lines = "\n".join(f"- {subject} : {count} 条" for subject, count in subject_counter.most_common()) or "- 暂无数据"
783|
784| # Calculate grade distribution
785| grade_counter = Counter(row.get("grade", "未知") for row in questions)
786| grade_lines = "\n".join(f"- {grade} : {count} 条" for grade, count in grade_counter.most_common()) or "- 暂无数据"
787|
788| # Correct vs wrong stats
789| correct = sum(1 for q in questions if q.get("student_answer") == q.get("answer"))
790| wrong = len(questions) - correct
791|
792| return f"""---
793| title: 版本说明
794| tags:
795| - project/k12-tracking
796| - copyright/materials
797| updated: {now_date()}
798| ---
799|
800| # 版本说明
801|
802| ## 一、当前版本
803|
804| 当前版本: v0.1.1 (首版草案)
805|
806| ## 二、版本特征
807|
808| ### 2.1 已完成能力
809|
810| - **手动录题闭环**：支持通过命令行手动录入题目，完成知识点映射、对错分流、掌握度更新。
811| - **照片导入骨架**：支持通过命令行传入照片路径和 OCR 文本，完成与手动录题相同的分析链路。
812| - **结果视图生成**：可生成家长端总览页、学生周报、题目详情页。
813| - **软著材料生成**：可自动生成系统说明、用户操作手册、安装说明、版本说明、日志索引。
814| - **审计日志**：所有关键操作写入 JSONL 日志，支持回溯。

```

815| -**完整知识体系**：15 学科 709 知识点，基于国家 2022 版课程标准。

816|

817| ### 2.2 后续计划

818|

819| - 接入真实 OCR（如 PaddleOCR、Tesseract）

820| - LLM 辅助知识点映射（当前基于规则匹配）

821| - 更多学科的双向链接和跨学科追踪

822| - 导出 PDF/Word 格式的软著申报材料

823|

824| ## 三、数据规模

825|

826| | 指标 | 数值 |

827| | :--- | :---: |

828| | 题目记录数 | {len(questions)} |

829| | 掌握状态记录数 | {len(mastery_rows)} |

830| | 导入记录数 | {len(ingest_rows)} |

831| | 正确题数 | {correct} |

832| | 错题数 | {wrong} |

833|

834| ## 四、学科分布

835|

836| {subject_lines}

837|

838| ## 五、年级分布

839|

840| {grade_lines}

841|

842| ## 六、知识体系

843|

844| | 指标 | 数值 |

845| | :--- | :---: |

846| | 覆盖学科 | 15 |

847| | 知识点卡片 | 709 |

848| | INDEX 文件 | 41 |

849| | 学段覆盖 | 小学（1-6 年级） 初中（7-9 年级） 高中 |

850|

851| ## 七、版本兼容性

852|

853| | 项目 | 版本 |

854| | :--- | :--- |

855| | Python | 3.9+ |

856| | macOS | 12+ |

857| | Obsidian | 推荐 v1.5+ |

858| """

859|

860|

861| def build_log_index(log_dir: Path) -> str:

862| files = sorted(p.name for p in log_dir.glob("*.jsonl"))

863| lines = [

864| "----",

865| "title: 日志索引",

866| "tags:",

867| " - project/k12-tracking",

868| " - copyright/materials",

869| f"updated: {now_date()}",

870| "----",

871| "",

872| "# 日志索引",

873| "",

874| "## 一、日志文件列表",

875| "",

876|]

877| total_size = 0

878| if files:

879| for name in files:

880| fp = log_dir / name

881| sz = fp.stat().st_size

882| total_size += sz

883| rec_count = len(fp.read_text().splitlines())

884| lines.append(f"- {name} - {sz:} bytes, {rec_count} 条记录")

885| else:

886| lines.append("- 暂无日志文件。")

887|

888| lines += [

```

889| """
890| "## 二、各日志用途",
891| """
892| "| 文件 | 记录内容 |",
893| "| :--- | :--- |",
894| "| `questions.jsonl` | 每题一比记录，含题目文字、答案、学生答案、映射知识点 |",
895| "| `mastery.jsonl` | 知识点掌握度快照，每个知识点在各个时间点的掌握概率 |",
896| "| `evidence.jsonl` | 证据事件，记录每次掌握度变化的原因和来源 |",
897| "| `ingest.jsonl` | 导入记录，包含原始输入参数和时间戳 |",
898| "| `audit.jsonl` | 审计日志，记录每次关键操作的完整上下文 |",
899| """
900| "## 三、数据总览",
901| """
902| f"- 日志总大小: {total_size:} bytes",
903| f"- 日志文件数: {len(files)}",
904| """
905| "> 日志文件为追加写入的 JSONL 格式，每条记录独立一行，可直接用文本编辑器或 jq 解析。",
906| ]
907| return "\n".join(lines) + "\n"
908|
909|
910| def build_source_code(code_dir: Path, output_dir: Path, lines_per_page: int = 50) -> Path:
911| """Extract source code into A4-ready format for soft copyright submission.
912|
913| Chinese soft copyright requires first 30 pages and last 30 pages of source code.
914| At ~50 lines per A4 page, that's ~1,500 lines per section.
915| If total code is < 60 pages (~3,000 lines), submit ALL code.
916| """
917| # Collect all .py files in order
918| py_files = sorted(code_dir.rglob("*.py"))
919| py_files = [f for f in py_files if "__pycache__" not in str(f)]
920| py_files = sorted(py_files, key=lambda x: x.name)
921|
922| all_lines: list[str] = []
923| current_file_lines: list[str] = []
924|
925| for pf in py_files:
926| rel = str(pf.relative_to(code_dir))
927| all_lines.append(f"// {' '*70}")
928| all_lines.append(f"// 文件: {rel}")
929| all_lines.append(f"// {' '*70}")
930| code = pf.read_text(encoding="utf-8")
931| for line in code.splitlines():
932| all_lines.append(line)
933| all_lines.append("") # blank line between files
934|
935| total = len(all_lines)
936|
937| # Build output
938| out_lines = [
939| "---",
940| "title: 源代码",
941| "tags:",
942| " - project/k12-tracking",
943| " - copyright/materials",
944| f"updated: {now_date()}",
945| "---",
946| """
947| f"# 源代码",
948| """
949| f"## 总览",
950| """
951| f"- 代码文件: {len(py_files)} 个",
952| f"- 总行数: {total}",
953| f"- 每页约 {lines_per_page} 行 (A4 排版标准)",
954| f"- 换算页数: 约 {total // lines_per_page} 页",
955| """
956| ]
957|
958| if total <= lines_per_page * 60:
959| out_lines.append("> 代码总量不足 60 页 (A4 标准)，按软著申报要求提交全部代码。")
960| out_lines.append("")
961| out_lines.append("---")
962| out_lines.append("")

```

```

963| # Output ALL code with line numbers
964| for i, line in enumerate(all_lines, 1):
965| out_lines.append(f"{i:>5}| {escape_obsidian_link_syntax(line)}")
966| else:
967| out_lines.append("> 代码总量超过 60 页，提交首 30 页和末 30 页。")
968| out_lines.append("")
969| out_lines.append("## 前 30 页")
970| out_lines.append("")
971| first = all_lines[:lines_per_page * 30]
972| for i, line in enumerate(first, 1):
973| out_lines.append(f"{i:>5}| {escape_obsidian_link_syntax(line)}")
974| out_lines.append("")
975| out_lines.append("---")
976| out_lines.append("")
977| out_lines.append("## 后 30 页")
978| out_lines.append("")
979| last = all_lines[-lines_per_page * 30:]
980| offset = total - len(last)
981| for i, line in enumerate(last, 1):
982| out_lines.append(f"{offset + i:>5}| {escape_obsidian_link_syntax(line)}")
983|
984| path = output_dir / "源代码.md"
985| path.write_text("\n".join(out_lines) + "\n", encoding="utf-8")
986| return path
987|
988|
989| def escape_obsidian_link_syntax(line: str) -> str:
990| return line.replace("\\", "\\").replace("\n", "\\n")
991|
992|
993| def generate_materials(project_root: Path) -> List[Path]:
994| vault_root = project_root / "教育智能体"
995| output_dir = vault_root / "09-软著材料" / "生成稿"
996| output_dir.mkdir(parents=True, exist_ok=True)
997|
998| service_dir = project_root / "services" / "edu_tracker"
999|
1000| log_dir = vault_root / "logs"
1001| questions = load_jsonl(log_dir / "questions.jsonl")
1002| mastery_rows = load_jsonl(log_dir / "mastery.jsonl")
1003| ingest_rows = load_jsonl(log_dir / "ingest.jsonl")
1004|
1005| outputs = {
1006| "系统说明.md": build_system_overview(service_dir),
1007| "用户操作手册.md": build_user_manual(),
1008| "安装部署说明.md": build_installation_guide(),
1009| "版本说明.md": build_version_note(questions, mastery_rows, ingest_rows),
1010| "日志索引.md": build_log_index(log_dir),
1011| }
1012|
1013| written: List[Path] = []
1014| for filename, content in outputs.items():
1015| path = output_dir / filename
1016| path.write_text(content, encoding="utf-8")
1017| written.append(path)
1018| print(f" {filename} ({len(content.splitlines())} 行)")
1019|
1020| # Generate source code extract
1021| src_path = build_source_code(service_dir, output_dir)
1022| written.append(src_path)
1023| print(f" 源代码.md ({len(src_path.read_text().splitlines())} 行)")
1024|
1025| return written
1026|
1027| // =====
1028| // 文件: models.py
1029| // =====
1030| from __future__ import annotations
1031|
1032| from dataclasses import dataclass, field, asdict
1033| from datetime import datetime, timezone
1034| from typing import Any, Optional
1035|
1036|

```

```

1037| def now_iso() -> str:
1038| return datetime.now(timezone.utc).isoformat()
1039|
1040|
1041| @dataclass
1042| class KnowledgePoint:
1043| point_code: str
1044| subject: str
1045| subject_code: str
1046| stage: str
1047| stage_code: str
1048| grade: str
1049| topic: str
1050| topic_code: str
1051| standard_version: str
1052| tracking_mode: str
1053| prerequisites: list[str] = field(default_factory=list)
1054| related_points: list[str] = field(default_factory=list)
1055|
1056| def to_dict(self) -> dict[str, Any]:
1057| return asdict(self)
1058|
1059|
1060| @dataclass
1061| class QuestionRecord:
1062| question_id: str
1063| student_id: str
1064| source_type: str
1065| subject: str
1066| subject_code: str
1067| grade: str
1068| text: str
1069| answer: str = ""
1070| student_answer: str = ""
1071| is_correct: Optional[bool] = None
1072| review_required: bool = False
1073| created_at: str = field(default_factory=now_iso)
1074|
1075| def to_dict(self) -> dict[str, Any]:
1076| return asdict(self)
1077|
1078|
1079| @dataclass
1080| class EvidenceEvent:
1081| event_id: str
1082| question_id: str
1083| point_code: str
1084| evidence_type: str
1085| evidence_strength: str
1086| confidence: float
1087| model_version: str
1088| created_at: str = field(default_factory=now_iso)
1089|
1090| def to_dict(self) -> dict[str, Any]:
1091| return asdict(self)
1092|
1093|
1094| @dataclass
1095| class MasteryState:
1096| student_id: str
1097| point_code: str
1098| mastery_score: float
1099| mastery_level: str
1100| evidence_count: int
1101| negative_evidence_count: int
1102| review_required: bool
1103| last_updated_at: str = field(default_factory=now_iso)
1104| last_evidence_id: str = ""
1105|
1106| def to_dict(self) -> dict[str, Any]:
1107| return asdict(self)
1108|
1109|
1110| @dataclass

```

```

1111| class AuditEvent:
1112|     audit_id: str
1113|     event_type: str
1114|     actor: str
1115|     target_id: str
1116|     before: Optional[dict[str, Any]\]
1117|     after: Optional[dict[str, Any]\]
1118|     reason: str
1119|     created_at: str = field(default_factory=now_iso)
1120|
1121|     def to_dict(self) -> dict[str, Any]:
1122|         return asdict(self)
1123|
1124|
1125| @dataclass
1126| class IngestEvent:
1127|     ingest_id: str
1128|     student_id: str
1129|     source_type: str
1130|     image_path: str
1131|     status: str
1132|     ocr_text: str = ""
1133|     ocr_confidence: float = 0.0
1134|     created_at: str = field(default_factory=now_iso)
1135|
1136|     def to_dict(self) -> dict[str, Any]:
1137|         return asdict(self)
1138|
1139| // =====
1140| // 文件: pipeline.py
1141| // =====
1142| from __future__ import annotations
1143|
1144| import json
1145| from pathlib import Path
1146| from uuid import uuid4
1147| from typing import Optional
1148|
1149| from .knowledge_registry import match_question
1150| from .models import AuditEvent, EvidenceEvent, IngestEvent, MasteryState, QuestionRecord
1151|
1152|
1153| def mastery_level(score: float) -> str:
1154|     if score < 0.2:
1155|         return "未接触"
1156|     if score < 0.45:
1157|         return "初步掌握"
1158|     if score < 0.75:
1159|         return "基本掌握"
1160|     return "熟练掌握"
1161|
1162|
1163| def evidence_strength(confidence: float) -> str:
1164|     if confidence >= 0.85:
1165|         return "强"
1166|     if confidence >= 0.6:
1167|         return "中"
1168|     return "弱"
1169|
1170|
1171| class LocalPipeline:
1172|     def __init__(self, workspace_root):
1173|         self.workspace_root = Path(workspace_root)
1174|         self.log_dir = self.workspace_root / "logs"
1175|         self.log_dir.mkdir(parents=True, exist_ok=True)
1176|
1177|     def process_manual_question(
1178|         self,
1179|         *,
1180|         student_id: str,
1181|         subject: str,
1182|         grade: str,
1183|         text: str,
1184|         answer: str = "",

```



```

1185| student_answer: str = "",
1186| ) -> dict:
1187| return self._process_question(
1188| source_type="manual",
1189| student_id=student_id,
1190| subject=subject,
1191| grade=grade,
1192| text=text,
1193| answer=answer,
1194| student_answer=student_answer,
1195| image_path="",
1196| ocr_confidence=1.0,
1197| )
1198|
1199| def process_photo(
1200| self,
1201| *,
1202| student_id: str,
1203| subject: str,
1204| grade: str,
1205| image_path: str,
1206| answer: str = "",
1207| student_answer: str = "",
1208| ocr_text: str = "",
1209| ) -> dict:
1210| text, ingest = self._prepare_photo_text(student_id=student_id, image_path=image_path, ocr_text=ocr_text)
1211| if not text:
1212| self._append_jsonl("ingest.jsonl", ingest.to_dict())
1213| return {
1214| "ingest": ingest.to_dict(),
1215| "status": "待解析",
1216| "reason": "未提供 OCR 文本，也未找到同名旁路文本文件",
1217| }
1218|
1219| result = self._process_question(
1220| source_type="photo",
1221| student_id=student_id,
1222| subject=subject,
1223| grade=grade,
1224| text=text,
1225| answer=answer,
1226| student_answer=student_answer,
1227| image_path=image_path,
1228| ocr_confidence=ingest.ocr_confidence,
1229| )
1230| result["ingest"] = ingest.to_dict()
1231| return result
1232|
1233| def _process_question(
1234| self,
1235| *,
1236| source_type: str,
1237| student_id: str,
1238| subject: str,
1239| grade: str,
1240| text: str,
1241| answer: str,
1242| student_answer: str,
1243| image_path: str,
1244| ocr_confidence: float,
1245| ) -> dict:
1246| question = QuestionRecord(
1247| question_id=f"q_{uuid4().hex[:12]}",
1248| student_id=student_id,
1249| source_type=source_type,
1250| subject=subject,
1251| subject_code=self._subject_code(subject),
1252| grade=grade,
1253| text=text,
1254| answer=answer,
1255| student_answer=student_answer,
1256| )
1257|
1258| match = match_question(subject, grade, text)

```

```

1259| is_correct = self._derive_correctness(answer, student_answer)
1260| question.is_correct = is_correct
1261| question.review_required = match.confidence < 0.6 or ocr_confidence < 0.6 or is_correct is None
1262|
1263| event = EvidenceEvent(
1264|     event_id=f"e_{uuid4().hex[:12]}",
1265|     question_id=question.question_id,
1266|     point_code=match.point.point_code,
1267|     evidence_type=f"{source_type}_question",
1268|     evidence_strength=evidence_strength(min(match.confidence, ocr_confidence)),
1269|     confidence=min(match.confidence, ocr_confidence),
1270|     model_version="rule-v1",
1271| )
1272|
1273| previous = self._latest_mastery_state(student_id, match.point.point_code)
1274| updated = self._update_mastery(previous, question, event)
1275| audit = AuditEvent(
1276|     audit_id=f"a_{uuid4().hex[:12]}",
1277|     event_type=f"{source_type}_pipeline",
1278|     actor="pipeline",
1279|     target_id=question.question_id,
1280|     before=previous.to_dict(),
1281|     after=updated.to_dict(),
1282|     reason=f"match={match.reason}; confidence={match.confidence:.2f}; ocr_confidence={ocr_confidence:.2f}",
1283| )
1284|
1285| if source_type == "photo":
1286|     ingest = IngestEvent(
1287|         ingest_id=f"i_{uuid4().hex[:12]}",
1288|         student_id=student_id,
1289|         source_type=source_type,
1290|         image_path=image_path,
1291|         status="已解析",
1292|         ocr_text=text,
1293|         ocr_confidence=ocr_confidence,
1294|     )
1295|     self._append_jsonl("ingest.jsonl", ingest.to_dict())
1296|
1297|     self._append_jsonl("questions.jsonl", question.to_dict())
1298|     self._append_jsonl("evidence.jsonl", event.to_dict())
1299|     self._append_jsonl("mastery.jsonl", updated.to_dict())
1300|     self._append_jsonl("audit.jsonl", audit.to_dict())
1301|
1302|     return {
1303|         "question": question.to_dict(),
1304|         "match": {
1305|             "point_code": match.point.point_code,
1306|             "subject": match.point.subject,
1307|             "grade": match.point.grade,
1308|             "topic": match.point.topic,
1309|             "confidence": match.confidence,
1310|             "reason": match.reason,
1311|         },
1312|         "ocr": {
1313|             "confidence": ocr_confidence,
1314|             "source_type": source_type,
1315|         },
1316|         "evidence": event.to_dict(),
1317|         "mastery": updated.to_dict(),
1318|         "audit": audit.to_dict(),
1319|     }
1320|
1321| def _update_mastery(
1322|     self,
1323|     previous: MasteryState,
1324|     question: QuestionRecord,
1325|     event: EvidenceEvent,
1326| ) -> MasteryState:
1327|     score = previous.mastery_score
1328|     if question.is_correct is True:
1329|         if event.evidence_strength == "强":
1330|             score += 0.35
1331|         elif event.evidence_strength == "中":
1332|             score += 0.2

```

```

1333| else:
1334|     score += 0.05
1335| elif question.is_correct is False:
1336| if event.evidence_strength=="强":
1337|     score -= 0.25
1338| elif event.evidence_strength=="中":
1339|     score -= 0.15
1340| else:
1341|     score -= 0.05
1342|
1343| score = max(0.0, min(1.0, score))
1344| return MasteryState(
1345|     student_id=previous.student_id,
1346|     point_code=previous.point_code,
1347|     mastery_score=round(score, 3),
1348|     mastery_level=mastery_level(score),
1349|     evidence_count=previous.evidence_count + 1,
1350|     negative_evidence_count=previous.negative_evidence_count + (1 if question.is_correct is False else 0),
1351|     review_required=previous.review_required or event.evidence_strength=="弱" or question.review_required,
1352|     last_evidence_id=event.event_id,
1353| )
1354|
1355| def _latest_mastery_state(self, student_id: str, point_code: str) -> MasteryState:
1356| latest: Optional[dict] = None
1357| path = self.log_dir / "mastery.jsonl"
1358| if path.exists():
1359| with path.open("r", encoding="utf-8") as handle:
1360| for line in handle:
1361| line = line.strip()
1362| if not line:
1363| continue
1364| row = json.loads(line)
1365| if row.get("student_id") != student_id or row.get("point_code") != point_code:
1366| continue
1367| if latest is None or row.get("last_updated_at", "") >= latest.get("last_updated_at", ""):
1368| latest = row
1369| if latest is None:
1370| return MasteryState(
1371|     student_id=student_id,
1372|     point_code=point_code,
1373|     mastery_score=0.0,
1374|     mastery_level="未接触",
1375|     evidence_count=0,
1376|     negative_evidence_count=0,
1377|     review_required=False,
1378| )
1379| return MasteryState(
1380|     student_id=student_id,
1381|     point_code=point_code,
1382|     mastery_score=float(latest.get("mastery_score", 0.0)),
1383|     mastery_level=str(latest.get("mastery_level", "未接触")),
1384|     evidence_count=int(latest.get("evidence_count", 0)),
1385|     negative_evidence_count=int(latest.get("negative_evidence_count", 0)),
1386|     review_required=bool(latest.get("review_required", False)),
1387|     last_updated_at=str(latest.get("last_updated_at", "")),
1388|     last_evidence_id=str(latest.get("last_evidence_id", "")),
1389| )
1390|
1391| def _append_jsonl(self, filename: str, payload: dict) -> None:
1392| path = self.log_dir / filename
1393| with path.open("a", encoding="utf-8") as f:
1394| f.write(json.dumps(payload, ensure_ascii=False) + "\n")
1395|
1396| def _subject_code(self, subject: str) -> str:
1397| return {"数学": "MATH", "语文": "CHN", "英语": "ENG"}.get(subject, "UNK")
1398|
1399| def _derive_correctness(self, answer: str, student_answer: str) -> Optional[bool]:
1400| if not answer and not student_answer:
1401| return None
1402| if not answer:
1403| return None
1404| return answer.strip() == student_answer.strip()
1405|
1406| def _prepare_photo_text(self, *, student_id: str, image_path: str, ocr_text: str) -> tuple[str, IngestEvent]:

```

```

1407| candidate_text = ocr_text.strip()
1408| sidecar = self._find_sidecar_text(image_path)
1409| if not candidate_text and sidecar is not None:
1410| candidate_text = sidecar.read_text(encoding="utf-8").strip()
1411| ingest = IngestEvent(
1412| ingest_id=f"i_{uuid4().hex[:12]}",
1413| student_id=student_id,
1414| source_type="photo",
1415| image_path=image_path,
1416| status="已解析" if candidate_text else "待解析",
1417| ocr_text=candidate_text,
1418| ocr_confidence=0.78 if candidate_text else 0.0,
1419| )
1420| return candidate_text, ingest
1421|
1422| def _find_sidecar_text(self, image_path: str):
1423| image = Path(image_path)
1424| candidates = [
1425| image.with_suffix(".txt"),
1426| image.parent / f"{image.stem}.txt",
1427| ]
1428| for candidate in candidates:
1429| if candidate.exists():
1430| return candidate
1431| return None
1432|
1433| // =====
1434| // 文件: reporting.py
1435| // =====
1436| from __future__ import annotations
1437|
1438| import json
1439| from collections import Counter, defaultdict
1440| from datetime import datetime, timezone
1441| from pathlib import Path
1442| import re
1443| from typing import Dict, Iterable, List, Optional, Tuple
1444|
1445| from .knowledge_registry import registry
1446|
1447|
1448| def now_date() -> str:
1449| return datetime.now(timezone.utc).strftime("%Y-%m-%d")
1450|
1451|
1452| def load_jsonl(path: Path) -> List[dict]:
1453| if not path.exists():
1454| return []
1455| rows: List[dict] = []
1456| with path.open("r", encoding="utf-8") as handle:
1457| for line in handle:
1458| line = line.strip()
1459| if not line:
1460| continue
1461| rows.append(json.loads(line))
1462| return rows
1463|
1464|
1465| def point_catalog() -> Dict[str, dict]:
1466| catalog: Dict[str, dict] = {}
1467| for point in registry():
1468| catalog[point.point_code] = point.to_dict()
1469| return catalog
1470|
1471|
1472| def build_point_note_index(vault_root: Path) -> Dict[str, dict]:
1473| tree_root = vault_root / "02-课标与知识体系" / "02-国家课标知识树"
1474| index: Dict[str, dict] = {}
1475| if not tree_root.exists():
1476| return index
1477| for path in tree_root.rglob("*.md"):
1478| meta = read_frontmatter(path)
1479| point_code = meta.get("point_code")
1480| if not point_code:

```

```

1481| continue
1482| title = extract_first_heading(path) or str(meta.get("title") or path.stem)
1483| index[point_code] = {
1484| "path": path.relative_to(vault_root).as_posix(),
1485| "abs_path": path,
1486| "title": title,
1487| }
1488| return index
1489|
1490|
1491| def read_frontmatter(path: Path) -> Dict[str, str]:
1492| text = path.read_text(encoding="utf-8")
1493| lines = text.splitlines()
1494| if not lines or lines[0].strip() != "---":
1495| return {}
1496| meta: Dict[str, str] = {}
1497| for line in lines[1:]:
1498| if line.strip() == "---":
1499| break
1500| match = re.match(r"^(?=[A-Za-z0-9_]+):s*(.*)$", line)
1501| if not match:
1502| continue
1503| key = match.group(1).strip()
1504| value = match.group(2).strip().strip("'").strip('"')
1505| meta[key] = value
1506| return meta
1507|
1508|
1509| def extract_first_heading(path: Path) -> Optional[str]:
1510| text = path.read_text(encoding="utf-8")
1511| lines = text.splitlines()
1512| in_frontmatter = False
1513| for line in lines:
1514| if line.strip() == "---":
1515| in_frontmatter = not in_frontmatter
1516| continue
1517| if in_frontmatter:
1518| continue
1519| stripped = line.strip()
1520| if stripped.startswith("# "):
1521| return stripped[2:].strip()
1522| return None
1523|
1524|
1525| def wikilink(path: str, alias: Optional[str] = None) -> str:
1526| return f"\[{path}\]{alias}\]" if alias else f"\[{path}\]"
1527|
1528|
1529| def question_detail_relative_path(question_id: str) -> str:
1530| return f"05-结果视图/题目详情/{question_id}.md"
1531|
1532|
1533| def student_report_relative_path(student_id: str) -> str:
1534| return f"05-结果视图/学生端-{student_id}.md"
1535|
1536|
1537| def latest_mastery(rows: Iterable[dict]) -> Dict[Tuple[str, str], dict]:
1538| latest: Dict[Tuple[str, str], dict] = {}
1539| for row in rows:
1540| key = (row.get("student_id", ""), row.get("point_code", ""))
1541| previous = latest.get(key)
1542| if previous is None or row.get("last_updated_at", "") >= previous.get("last_updated_at", ""):
1543| latest[key] = row
1544| return latest
1545|
1546|
1547| def render_student_report(
1548| *,
1549| student_id: str,
1550| questions: List[dict],
1551| evidence_rows: List[dict],
1552| mastery_rows: List[dict],
1553| ingest_rows: List[dict],
1554| point_index: Dict[str, dict],

```

```

1555| ) -> str:
1556| catalog = point_catalog()
1557| student_questions = [row for row in questions if row.get("student_id") == student_id]
1558| student_mastery = [row for row in mastery_rows if row.get("student_id") == student_id]
1559| student_ingest = [row for row in ingest_rows if row.get("student_id") == student_id]
1560| question_to_point = {
1561|     row.get("question_id", ""): row.get("point_code", "")
1562|     for row in evidence_rows
1563|     if row.get("question_id") and row.get("point_code")
1564| }
1565| latest = latest_mastery(student_mastery)
1566|
1567| mastery_list = sorted(
1568|     latest.values(),
1569|     key=lambda row: (row.get("mastery_score", 0.0), row.get("negative_evidence_count", 0), row.get("point_code", "")),
1570| )
1571| shortboards = [row for row in mastery_list if row.get("mastery_score", 0.0) < 0.75 or row.get("review_required")]
1572| recent_questions = student_questions[-5:]
1573|
1574| subject_counter = Counter(row.get("subject", "未知") for row in student_questions)
1575| review_counter = Counter(row.get("status", "") for row in student_ingest)
1576|
1577| lines: List[str] = []
1578| lines.append(f"---")
1579| lines.append(f"title: 学生端周报-{student_id}")
1580| lines.append(f"tags:")
1581| lines.append(f" - project/k12-tracking")
1582| lines.append(f" - result-view/student")
1583| lines.append(f"student_id: {student_id}")
1584| lines.append(f"updated: {now_date()}")
1585| lines.append(f"---")
1586| lines.append("")
1587| lines.append(f"# 学生端周报 - {student_id}")
1588| lines.append("")
1589| lines.append("## 总览")
1590| lines.append("")
1591| lines.append(f"- 本周录入题目数: {len(student_questions)}")
1592| lines.append(f"- 本周覆盖学科数: {len(subject_counter)}")
1593| lines.append(f"- 本周照片导入数: {len(student_ingest)}")
1594| lines.append(f"- 需要复核的记录数: {sum(1 for row in mastery_list if row.get('review_required'))}")
1595| lines.append("")
1596| lines.append("## 学科分布")
1597| lines.append("")
1598| if subject_counter:
1599| lines.append("| 学科 | 题目数 |")
1600| lines.append("| --- | --- |")
1601| for subject, count in subject_counter.most_common():
1602| lines.append(f"| {subject} | {count} |")
1603| else:
1604| lines.append("- 暂无录入记录。")
1605| lines.append("")
1606| lines.append("## 短板清单")
1607| lines.append("")
1608| if shortboards:
1609| lines.append("| 知识点代码 | 知识点名称 | 知识点页面 | 掌握度 | 等级 | 证据数 | 复核 |")
1610| lines.append("| --- | --- | --- | --- | --- | --- | --- |")
1611| for row in shortboards[:5]:
1612| point_code = row.get("point_code", "")
1613| point = catalog.get(point_code, {})
1614| point_meta = point_index.get(point_code, {})
1615| point_title = point_meta.get("title") or point.get("topic", point_code)
1616| point_link = wikilink(point_meta["path"], point_meta["title"]) if point_meta else point_code
1617| lines.append(
1618|     f"| {point_code} | {point_title} | {point_link} | {row.get('mastery_score', 0.0)} | "
1619|     f"{row.get('mastery_level', '')} | {row.get('evidence_count', 0)} | {'是' if row.get('review_required') else '否'} |"
1620| )
1621| else:
1622| lines.append("- 当前没有明显短板。")
1623| lines.append("")
1624|
1625| linked_point_codes = []
1626| seen_points = set()
1627| for row in student_questions:
1628| point_code = question_to_point.get(row.get("question_id", ""), "")

```

```

1629| if not point_code or point_code in seen_points:
1630|     continue
1631|     seen_points.add(point_code)
1632|     linked_point_codes.append(point_code)
1633|     if linked_point_codes:
1634|         lines.append("## 关联知识点")
1635|         lines.append("")
1636|         for point_code in linked_point_codes:
1637|             point_meta = point_index.get(point_code, {})
1638|             if point_meta:
1639|                 lines.append(f"- {wikilink(point_meta['path'], point_meta['title'])} `({point_code})`")
1640|             else:
1641|                 lines.append(f"- `{point_code}`")
1642|         lines.append("")
1643|
1644| lines.append("## 最近题目")
1645| lines.append("")
1646| if recent_questions:
1647| lines.append("| 时间 | 学科 | 年级 | 内容 | 关联知识点 | 题目详情 | 结果 |")
1648| lines.append("| --- | --- | --- | --- | --- | --- | --- |")
1649| for row in recent_questions:
1650| result = "正确" if row.get("is_correct") is True else "错误" if row.get("is_correct") is False else "待判断"
1651| point_code = question_to_point.get(row.get("question_id", ""), "")
1652| point_meta = point_index.get(point_code, {})
1653| point_link = wikilink(point_meta["path"], point_meta["title"]) if point_meta else point_code
1654| detail_link = wikilink(question_detail_relative_path(row.get("question_id", "")), "查看")
1655| lines.append(
1656| f"| {row.get('created_at', '')} | {row.get('subject', '')} | {row.get('grade', '')} | "
1657| f"{row.get('text', '')} | {point_link} | {detail_link} | {result} | "
1658| )
1659| else:
1660| lines.append("- 暂无最近题目。")
1661| lines.append("")
1662| lines.append("## 本周提醒")
1663| lines.append("")
1664| if review_counter.get("待解析", 0) or review_counter.get("待复核", 0):
1665| lines.append("- 有待处理的导入或复核记录，建议先清理。")
1666| if shortboards:
1667| lines.append("- 优先复习短板清单中的前 3 个知识点。")
1668| if not shortboards and not recent_questions:
1669| lines.append("- 当前暂无可展示的学习记录。")
1670| return "\n".join(lines) + "\n"
1671|
1672|
1673| def render_parent_overview(student_reports: List[Tuple[str, str]], total_questions: int, total_students: int) -> str:
1674| lines: List[str] = []
1675| lines.append("---")
1676| lines.append("title: 家长端总览")
1677| lines.append("tags:")
1678| lines.append(" - project/k12-tracking")
1679| lines.append(" - result-view/parent")
1680| lines.append(f"updated: {now_date()}")
1681| lines.append("---")
1682| lines.append("")
1683| lines.append("# 家长端总览")
1684| lines.append("")
1685| lines.append("## 总体情况")
1686| lines.append("")
1687| lines.append(f"- 覆盖学生数: {total_students}")
1688| lines.append(f"- 已记录题目数: {total_questions}")
1689| lines.append("")
1690| lines.append("## 学生周报")
1691| lines.append("")
1692| if student_reports:
1693| for student_id, relative_path in student_reports:
1694| lines.append(f"- \[{relative_path}\] {student_id} 周报\]")
1695| else:
1696| lines.append("- 暂无学生周报。")
1697| lines.append("")
1698| lines.append("## 使用建议")
1699| lines.append("")
1700| lines.append("- 优先查看短板清单前 3 项。")
1701| lines.append("- 关注带有复核标记的记录，避免直接下结论。")
1702| lines.append("- 结果页只展示中文摘要和必要证据，不展开内部实现细节。")

```

```

1703| return "\n".join(lines) + "\n"
1704|
1705|
1706| def generate_reports(project_root: Path, student_id: Optional[str] = None) -> List[Path]:
1707| vault_root=project_root/"教育智能体"
1708| log_dir = vault_root / "logs"
1709| output_dir=vault_root/"05-结果视图"
1710| detail_dir=output_dir/"题目详情"
1711| output_dir.mkdir(parents=True, exist_ok=True)
1712| detail_dir.mkdir(parents=True, exist_ok=True)
1713| point_index = build_point_note_index(vault_root)
1714|
1715| questions = load_jsonl(log_dir / "questions.jsonl")
1716| evidence_rows = load_jsonl(log_dir / "evidence.jsonl")
1717| mastery_rows = load_jsonl(log_dir / "mastery.jsonl")
1718| audit_rows = load_jsonl(log_dir / "audit.jsonl")
1719| ingest_rows = load_jsonl(log_dir / "ingest.jsonl")
1720| evidence_by_question = {row.get("question_id", ""): row for row in evidence_rows if row.get("question_id")}
1721| audit_by_question = {row.get("target_id", ""): row for row in audit_rows if row.get("target_id")}
1722| latest_mastery_by_point = latest_mastery(mastery_rows)
1723|
1724| if student_id:
1725| student_ids = [student_id]
1726| else:
1727| student_ids = sorted({row.get("student_id", "") for row in questions if row.get("student_id")})
1728|
1729| generated: List[Path] = []
1730| student_refs: List[Tuple[str, str]\] = []
1731| for sid in student_ids:
1732| student_questions = [row for row in questions if row.get("student_id") == sid]
1733| for row in student_questions:
1734| question_id = row.get("question_id", "")
1735| point_code = evidence_by_question.get(question_id, {}).get("point_code", "")
1736| point_meta = point_index.get(point_code, {})
1737| mastery_snapshot = latest_mastery_by_point.get((sid, point_code), {})
1738| detail_text = render_question_detail(
1739| question=row,
1740| point_code=point_code,
1741| point_meta=point_meta,
1742| point_index=point_index,
1743| evidence_row=evidence_by_question.get(question_id, {}),
1744| mastery_row=mastery_snapshot,
1745| audit_row=audit_by_question.get(question_id, {}),
1746| student_report_path=student_report_relative_path(sid),
1747| )
1748| detail_path = detail_dir / f"{question_id}.md"
1749| detail_path.write_text(detail_text, encoding="utf-8")
1750| generated.append(detail_path)
1751| report_text = render_student_report(
1752| student_id=sid,
1753| questions=questions,
1754| evidence_rows=evidence_rows,
1755| mastery_rows=mastery_rows,
1756| ingest_rows=ingest_rows,
1757| point_index=point_index,
1758| )
1759| relative_name = f"05-结果视图/学生端-{sid}.md"
1760| report_path = output_dir / f"学生端-{sid}.md"
1761| report_path.write_text(report_text, encoding="utf-8")
1762| generated.append(report_path)
1763| student_refs.append((sid, relative_name))
1764|
1765| parent_path = output_dir / "家长端总览.md"
1766| parent_text = render_parent_overview(student_refs, len(questions), len(student_ids))
1767| parent_path.write_text(parent_text, encoding="utf-8")
1768| generated.insert(0, parent_path)
1769| return generated
1770|
1771|
1772| def render_question_detail(
1773| *,
1774| question: dict,
1775| point_code: str,
1776| point_meta: Dict[str, dict],

```



```

1777| point_index: Dict[str, dict],
1778| evidence_row: dict,
1779| mastery_row: dict,
1780| audit_row: dict,
1781| student_report_path: str,
1782| ) -> str:
1783| catalog = point_catalog()
1784| point_data = catalog.get(point_code, {})
1785| question_id = question.get("question_id", "")
1786| point_link = ""
1787| if point_meta:
1788| point_link = wikilink(point_meta["path"], point_meta["title"])
1789| elif point_code:
1790| point_link = point_code
1791| else:
1792| point_link = "未识别"
1793| answer = question.get("answer", "")
1794| student_answer = question.get("student_answer", "")
1795| result = "正确" if question.get("is_correct") is True else "错误" if question.get("is_correct") is False else "待判断"
1796| review_required = "是" if question.get("review_required") else "否"
1797| error_reason = infer_error_reason(question, point_data, evidence_row)
1798| recommendation = build_recommendation(point_data, point_meta, question, evidence_row, mastery_row)
1799| result_is_correct = question.get("is_correct") is True
1800| result_is_wrong = question.get("is_correct") is False
1801|
1802| lines: List[str] = []
1803| lines.append("---")
1804| lines.append(f"title: 题目详情-{question_id}")
1805| lines.append("tags:")
1806| lines.append(" - project/k12-tracking")
1807| lines.append(" - result-view/question")
1808| lines.append(f"question_id: {question_id}")
1809| lines.append(f"student_id: {question.get('student_id', '')}")
1810| lines.append(f"point_code: {point_code}")
1811| lines.append(f"updated: {now_date()}")
1812| lines.append("---")
1813| lines.append("")
1814| lines.append(f"# 题目详情 - {question_id}")
1815| lines.append("")
1816| lines.append("## 基本信息")
1817| lines.append("")
1818| lines.append(f"- 学生: {question.get('student_id', '')}")
1819| lines.append(f"- 学科: {question.get('subject', '')}")
1820| lines.append(f"- 年级: {question.get('grade', '')}")
1821| lines.append(f"- 来源: {question.get('source_type', '')}")
1822| lines.append(f"- 结果: {result}")
1823| lines.append(f"- 复核: {review_required}")
1824| lines.append(f"- 回到周报: \[{student_report_path}| 学生周报\]")
1825| lines.append("")
1826| lines.append("## 原题")
1827| lines.append("")
1828| lines.append(question.get("text", "") or "无")
1829| lines.append("")
1830| lines.append("## 作答信息")
1831| lines.append("")
1832| lines.append(f"- 标准答案: {answer or '无'}")
1833| lines.append(f"- 学生作答: {student_answer or '无'}")
1834| lines.append(f"- 关联知识点: {point_link}")
1835| lines.append("")
1836| lines.append("## 证据与掌握")
1837| lines.append("")
1838| lines.append(f"- 证据强度: {evidence_row.get('evidence_strength', '未知')}")
1839| lines.append(f"- 置信度: {evidence_row.get('confidence', '')}")
1840| lines.append(f"- 掌握度: {mastery_row.get('mastery_score', '')}")
1841| lines.append(f"- 掌握等级: {mastery_row.get('mastery_level', '')}")
1842| lines.append(f"- 最近证据: {mastery_row.get('last_evidence_id', '')}")
1843| lines.append(f"- 复核建议: {'需要' if mastery_row.get('review_required') else '不需要'}")
1844| lines.append(f"- 错因判断: {error_reason}")
1845| lines.append("")
1846| if result_is_correct:
1847| lines.append("## 本题结论")
1848| lines.append("")
1849| lines.append("- 本题作答正确。")
1850| lines.append("- 这类题可作为正向证据，但仍要结合整体掌握度判断是否继续巩固。")

```

```

1851| lines.append("")
1852| elif result_is_wrong:
1853| lines.append("## 错题分析")
1854| lines.append("")
1855| lines.append("- 本题作答错误。")
1856| lines.append(f"- 主要错因：{error_reason}")
1857| lines.append("")
1858| lines.append("## 纠错重点")
1859| lines.append("")
1860| lines.append("- 先复核题干、标准答案和学生作答。")
1861| lines.append("- 再回到对应知识点的前置知识与基础题重新练习。")
1862| lines.append("")
1863| lines.append("## 下次复习")
1864| lines.append("")
1865| lines.append(f"- {recommendation}")
1866| lines.append("")
1867| else:
1868| lines.append("## 本题结论")
1869| lines.append("")
1870| lines.append("- 当前无法明确判断对错。")
1871| lines.append(f"- 建议：{recommendation}")
1872| lines.append("")
1873| lines.append("## 前置知识")
1874| lines.append("")
1875| prerequisites = extract_section_bullets(
1876| Path(point_meta["abs_path"]) if point_meta and point_meta.get("abs_path") else None,
1877| "## 前置知识",
1878| )
1879| if not prerequisites:
1880| prerequisites = point_data.get("prerequisites", []) or []
1881| if prerequisites:
1882| for item in prerequisites:
1883| lines.append(f"- {item}")
1884| else:
1885| lines.append("- 暂无前置知识记录。")
1886| lines.append("")
1887| lines.append("## 复习建议")
1888| lines.append("")
1889| if result_is_wrong:
1890| lines.append("- 错题优先补基础，再做同类变式题。")
1891| elif result_is_correct:
1892| lines.append("- 正确题可做少量同类巩固题，保持熟练度。")
1893| else:
1894| lines.append(f"- {recommendation}")
1895| lines.append("")
1896| lines.append("## 审计记录")
1897| lines.append("")
1898| if audit_row:
1899| lines.append(f"- 审计类型：{audit_row.get('event_type', '')}")
1900| lines.append(f"- 审计原因：{audit_row.get('reason', '')}")
1901| else:
1902| lines.append("- 暂无审计记录。")
1903| lines.append("")
1904| lines.append("## 关联知识点")
1905| lines.append("")
1906| if point_code and point_meta:
1907| lines.append(f"- {wikilink(point_meta['path'], point_meta['title'])}")
1908| lines.append(f"- 关联知识点代码：`{point_code}`")
1909| elif point_code:
1910| lines.append(f"- ``{point_code}``")
1911| else:
1912| lines.append("- 暂未识别出知识点。")
1913| lines.append("")
1914| lines.append("## 反向追踪")
1915| lines.append("")
1916| lines.append("- 题目详情页可通过知识点页的反向链接被检索到。")
1917| lines.append("- 周报页也会链接到本页，便于在 Obsidian 中往返查看。")
1918| return "\n".join(lines) + "\n"
1919|
1920|
1921| def infer_error_reason(question: dict, point_data: dict, evidence_row: dict) -> str:
1922| if question.get("is_correct") is True:
1923| return "本题作答正确，暂未发现明显错因。"
1924| if question.get("is_correct") is None:

```

```

1925| return "答案信息不足，暂无法判断错因。"
1926| if evidence_row.get("evidence_strength") == "弱":
1927| return "证据较弱，建议先复核题干与答案，再判断是否为知识点错误。"
1928| topic = str(point_data.get("topic", ""))
1929| topic_code = str(point_data.get("topic_code", ""))
1930| if topic_code == "NS" or "数与代数" in topic:
1931| return "更像是计算步骤或口诀调用不稳定。"
1932| if topic_code == "READ" or "阅读" in topic:
1933| return "更像是关键信息提取或理解偏差。"
1934| if topic_code == "GRM" or "语法" in topic:
1935| return "更像是规则应用不稳定或句型转换失误。"
1936| return "更像是知识点理解不足或题目迁移能力不足。"
1937|
1938|
1939| def build_recommendation(
1940| point_data: dict,
1941| point_meta: dict,
1942| question: dict,
1943| evidence_row: dict,
1944| mastery_row: dict,
1945| ) -> str:
1946| point_code = str(point_data.get("point_code", ""))
1947| topic = str(point_meta.get("title") or point_data.get("topic", ""))
1948| if question.get("review_required") or evidence_row.get("evidence_strength") == "弱":
1949| return "先人工复核题目与答案，再重新判断是否需要更新掌握度。"
1950| if mastery_row.get("mastery_score", 0.0) < 0.45:
1951| return f"优先回到 {topic or point_code} 的基础练习题，做 3 到 5 道同类题巩固。"
1952| if mastery_row.get("mastery_score", 0.0) < 0.75:
1953| return f"继续做 {topic or point_code} 的变式题，重点检查步骤是否稳定。"
1954| return f"保持复习节奏，围绕 {topic or point_code} 做少量巩固题即可。"
1955|
1956|
1957| def extract_section_bullets(path: Optional[Path], heading: str) -> List[str]:
1958| if path is None or not path.exists():
1959| return []
1960| lines = path.read_text(encoding="utf-8").splitlines()
1961| target_index: Optional[int] = None
1962| for i, line in enumerate(lines):
1963| if line.strip() == heading:
1964| target_index = i
1965| break
1966| if target_index is None:
1967| return []
1968| bullets: List[str] = []
1969| for line in lines[target_index + 1 :]:
1970| stripped = line.strip()
1971| if stripped.startswith("## "):
1972| break
1973| if stripped.startswith("- "):
1974| bullets.append(stripped[2:].strip())
1975| return bullets
1976|

```